

Tool Calls

Tool Calls 让模型能够调用外部工具，来增强自身能力。

非思考模式

样例代码

这里以获取用户当前位置的天气信息为例，展示了使用 Tool Calls 的完整 Python 代码。

Tool Calls 的具体 API 格式请参考[对话补全](#)文档。

```
from openai import OpenAI

def send_messages(messages):
    response = client.chat.completions.create(
        model="deepseek-v4-pro",
        messages=messages,
        tools=tools
    )
    return response.choices[0].message

client = OpenAI(
    api_key="<your api key>",
    base_url="https://api.deepseek.com",
)

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather of a location, the user should supply a location first.",
            "parameters": {
                "type": "object",
                "properties": {
```

```

        "location": {
            "type": "string",
            "description": "The city and state, e.g. San
Francisco, CA",
        },
    },
    "required": ["location"]
},
]

messages = [{"role": "user", "content": "How's the weather in Hangzhou,
Zhejiang?"}]
message = send_messages(messages)
print(f"User>\t {messages[0]['content']}")

tool = message.tool_calls[0]
messages.append(message)

messages.append({"role": "tool", "tool_call_id": tool.id, "content":
"24°"})
message = send_messages(messages)
print(f"Model>\t {message.content}")

```

这个例子的执行流程如下：

1. 用户：询问现在的天气
2. 模型：返回 function `get_weather({location: 'Hangzhou'})`
3. 用户：调用 function `get_weather({location: 'Hangzhou'})`，并传给模型。
4. 模型：返回自然语言，"The current temperature in Hangzhou is 24°C."

注：上述代码中 `get_weather` 函数功能需由用户提供，模型本身不执行具体函数。

思考模式

从 DeepSeek-V3.2 开始，API 支持了思考模式下的工具调用能力，详见[思考模式](#)。

`strict` 模式 (Beta)

在 `strict` 模式下，模型在输出 Function 调用时会严格遵循 Function 的 JSON Schema 的格式要求，以确保模型输出的 Function 符合用户的定义。在思考与非思考模式下的工具调用，均可使用 `strict` 模式。

要使用 `strict` 模式，需要：

1. 用户需要设置 `base_url="https://api.deepseek.com/beta"` 来开启 Beta 功能
2. 在传入的 `tools` 列表中，所有 `function` 均需设置 `strict` 属性为 `true`
3. 服务端会对用户传入的 Function 的 JSON Schema 进行校验，如不符合规范，或遇到服务端不支持的 JSON Schema 类型，将返回错误信息

以下是 `strict` 模式下 tool 的定义样例：

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "strict": true,
    "description": "Get weather of a location, the user should supply a location first.",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "The city and state, e.g. San Francisco, CA",
        },
      },
      "required": ["location"],
      "additionalProperties": false
    }
  }
}
```

`strict` 模式支持的 JSON Schema 类型

- object
- string

- number
 - integer
 - boolean
 - array
 - enum
 - anyOf
-

object 类型

object 定义一个包含键值对的深层结构，其中 properties 定义了对象中每个键（属性）的 schema。每个 **object** 的所有属性均需设置为 **required**，且 **object** 中 **additionalProperties** 属性必须为 **false**。

示例：

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "age": { "type": "integer" }
  },
  "required": ["name", "age"],
  "additionalProperties": false
}
```

string 类型

- 支持的参数：
 - pattern：使用正则表达式来约束字符串的格式
 - format：使用预定义的常见格式进行校验，目前支持：
 - email：电子邮件地址
 - hostname：主机名
 - ipv4：IPv4 地址
 - ipv6：IPv6 地址
 - uuid：uuid

- 不支持的参数
 - minLength
 - maxLength

示例：

```
{
  "type": "object",
  "properties": {
    "user_email": {
      "type": "string",
      "description": "The user's email address",
      "format": "email"
    },
    "zip_code": {
      "type": "string",
      "description": "Six digit postal code",
      "pattern": "^\\d{6}$"
    }
  }
}
```

number/integer 类型

- 支持的参数
 - const: 固定数字为常数
 - default: 数字的默认值
 - minimum: 最小值
 - maximum: 最大值
 - exclusiveMinimum: 不小于
 - exclusiveMaximum: 不大于
 - multipleOf: 数字输出为这个值的倍数

示例：

```
{
  "type": "object",
  "properties": {
```

```
    "score": {
      "type": "integer",
      "description": "A number from 1-5, which represents your
rating, the higher, the better",
      "minimum": 1,
      "maximum": 5
    },
    "required": ["score"],
    "additionalProperties": false
  }
```

array 类型

- 不支持的参数
 - minItems
 - maxItems

示例:

```
{
  "type": "object",
  "properties": {
    "keywords": {
      "type": "array",
      "description": "Five keywords of the article, sorted by
importance",
      "items": {
        "type": "string",
        "description": "A concise and accurate keyword or phrase."
      }
    }
  },
  "required": ["keywords"],
  "additionalProperties": false
}
```

enum

enum 可以确保输出是预期的几个选项之一，例如在订单状态的场景下，只能是有限几个状态之一。

样例：

```
{
  "type": "object",
  "properties": {
    "order_status": {
      "type": "string",
      "description": "Ordering status",
      "enum": ["pending", "processing", "shipped", "cancelled"]
    }
  }
}
```

anyOf

匹配所提供的多个 schema 中的任意一个，可以处理可能具有多种有效格式的字段，例如用户的账户可能是邮箱或者手机号中的一个：

```
{
  "type": "object",
  "properties": {
    "account": {
      "anyOf": [
        { "type": "string", "format": "email", "description": "可以是电子邮件地址" },
        { "type": "string", "pattern": "^[\\d]{11}$", "description": "或11位手机号码" }
      ]
    }
  }
}
```

\$ref 和 \$def

可以使用 \$def 定义模块，再用 \$ref 引用以减少模式的重复和模块化，此外还可以单独使用 \$ref 定义递归结构。

```

{
  "type": "object",
  "properties": {
    "report_date": {
      "type": "string",
      "description": "The date when the report was published"
    },
    "authors": {
      "type": "array",
      "description": "The authors of the report",
      "items": {
        "$ref": "#/$def/author"
      }
    }
  },
  "required": ["report_date", "authors"],
  "additionalProperties": false,
  "$def": {
    "author": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "author's name"
        },
        "institution": {
          "type": "string",
          "description": "author's institution"
        },
        "email": {
          "type": "string",
          "format": "email",
          "description": "author's email"
        }
      },
      "additionalProperties": false,
      "required": ["name", "institution", "email"]
    }
  }
}

```